

Problem1:

(a)

$$n^{\log_2 \log_2 n} \quad (\log_2 n)^{\log_2 n} : A = \Theta(B)$$

$$2^{\sqrt{2 \log_2 n}} \quad \sqrt{n} : A = O(B)$$

$$n! \quad n^{n/2} : A = \Omega(B)$$

$$4^{\sqrt{n}} \quad 2^n : A = O(B)$$

$$\sum_{i=1}^n \frac{1}{i} \quad n \log n : A = O(B)$$

(b)

$$O(n^2)$$

Problem2:

Base case: $T(1) = 1$

$$T(n) = 2T\left(\frac{n}{2}\right) + \sqrt{n}$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \sqrt{\frac{n}{2}}$$

$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \sqrt{\frac{n}{2^2}}$$

$$\therefore T(n) = 2^2 T\left(\frac{n}{2^2}\right) + \sqrt{2}\sqrt{n} + \sqrt{n}$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + 2\sqrt{n} + \sqrt{2}\sqrt{n} + \sqrt{n}$$

...

after k substitutions:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sqrt{n}(1 + \sqrt{2} + 2 + \dots + \sqrt{2}^{(k-1)})$$

at last: $\frac{n}{2^k} = 1$

$$\therefore k = \log_2 n$$

$$\therefore T(n) = n + \sqrt{n}(1 + \sqrt{2} + 2 + \dots + \sqrt{2}^{(\log_2 n - 1)})$$

$$1 + \sqrt{2} + 2 + \dots + \sqrt{2}^{(\log_2 n - 1)} = \frac{\sqrt{2}^{(\log_2 n)} - 1}{\sqrt{2} - 1} = \frac{\sqrt{n} - 1}{\sqrt{2} - 1}$$

$$\therefore T(n) = n + \sqrt{n} \frac{\sqrt{n} - 1}{\sqrt{2} - 1}$$

$$\frac{\sqrt{n} - 1}{\sqrt{2} - 1} = O(\sqrt{n}) \quad \sqrt{n} \frac{\sqrt{n} - 1}{\sqrt{2} - 1} = O(n) \quad n + \sqrt{n} \frac{\sqrt{n} - 1}{\sqrt{2} - 1} = O(n)$$

$\therefore$ the tightest asymptotic upper bound is: $T(n) = O(n)$

Problem3:

(a)

For the outer loop: i goes from 1 to n $\therefore$ takes n steps

For the inner loop: j goes from 1 to limit (worst case k) $\therefore$ takes k steps

$\therefore$ the time complexity: $O(kn)$

(b)

For the first loop: m goes from 1 to n $\therefore$ takes n steps.

For the outer loop: i goes from 1 to n $\therefore$ takes n steps.

For the inner loop: j goes from i to n .

The total number of iterations is $n + (n - 1) + \dots + 1 = O(n^2)$.

$\therefore$ the total time complexity: $O(n) + O(n^2) = O(n^2)$.

(c)

FindMaxBoundedSubarray1: $O(kn)$

FindMaxBoundedSubarray2: $O(n^2)$

$\therefore$ when $k = O(n)$, FindMaxBoundedSubarray1 run strictly faster than FindMaxBoundedSubarray2

when $k = \Theta(n)$, they exhibit the same asymptotic runtime

Problem4:

(a)

Base case: when $n=1$, executes 3 baseline operations $\therefore T(1) = 3$

when $n>1$, there are two subproblems each with the scale of $n-1$. at the same time each call executes 3 constant-time boundary verification steps

$$\therefore T(n) = 2T(n-1) + 3$$

(b)

Base case: $T(1) = 3$

$$T(n) = 2T(n-1) + 3$$

$$T(n) = 4T(n-2) + 3 + 6$$

$$T(n) = 8T(n-3) + 3 + 6 + 12$$

$$T(n) = 2^k T(n-k) + 3(1+2+\dots+2^{k-1})$$

when it goes to $T(1)$: $n-k = 1 \quad k = n-1$

$$\therefore T(n) = 3 \cdot 2^{n-1} + 3(1+2+\dots+2^{n-2})$$

$$T(n) = 3(1+2+\dots+2^{n-2}+2^{n-1})$$

$$T(n) = 3 \cdot 2^n - 3$$

(c)

Base case: $T(1) = 3 \cdot 2 - 3 = 3$ correct

assume the expression is correct

For $n=k$:

$$T(k) = 3 \cdot 2^k - 3$$

$$T(k+1) = 3 \cdot 2^{k+1} - 3$$

$$T(k+1) = 2T(k) + 3$$

$\therefore$ correct